# แล็บ: เทรน VAE, GAN และ Diffusion ด้วยมือเราเอง

สัปดาห์ 4 · ช่วงบ่าย — Google Colab (T4) · ชุดข้อมูล Sprites 16×16

ดร.ธีรพงศ์ อรชร · ANT-DPU

ภาพรวม

# บ่ายนี้: notebook ไฟล์เดียว 4 แล็บ ชุดข้อมูลเดียว

### 1 แล็บ 1 · VAE

สร้างโมเดล → เทรน 15 epoch
ดูภาพคลายกลับ / สุ่มใหม่ / interpolate แผนที่ latent

### 2 แล็บ 2 · GAN

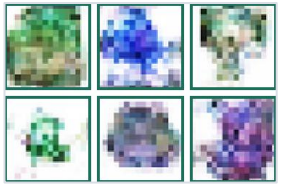
สร้าง G, D → เทรน 25 epoch
ดู loss แกว่ง + ภาพทุก 5 epoch
เทียบ VAE vs GAN + วัดความหลากหลาย

### 3 แล็บ 3 · DDPM / DDIM

ดู forward noising
เทรน U-Net 30 epoch
sample 1,000 ก้าว → DDIM 50 / 20 / 10

### 4 แล็บ 4 · เงื่อนไข

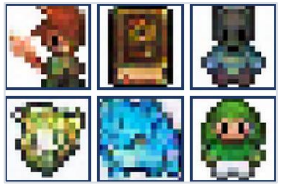
U-Net รับป้ายกลุ่ม → เทรน 30 epoch
สั่งสร้างทีละกลุ่ม หมุน guidance 1 / 3 / 6



แล็บ 1



แล็บ 2



แล็บ 3

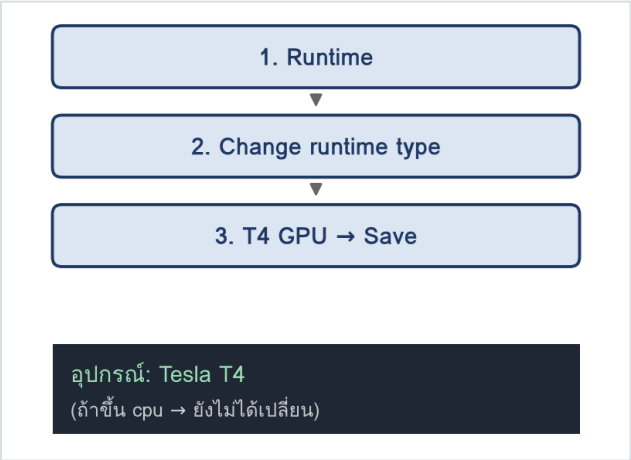


แล็บ 4

*ผลจริงจากการรัน notebook แล็บ (CPU)*

# ข้อตกลงก่อนเริ่ม

- **Runtime → Change runtime type → T4 GPU — เซลล์แรกจะพิมพ์ชื่อ GPU ถ้าขึ้น cpu ให้เปลี่ยนก่อน**
- รันจากบนลงล่างทีละเซลล์ ไม่ข้าม — เซลล์หลังใช้ตัวแปรจากเซลล์ก่อน (loader, show_grid, diversity, sample)
- ข้อมูลฝังอยู่ในเซลล์ 0.2 (base64) ไม่ต้องอัปโหลดอะไร · ไลบรารีเพิ่มแค่ diffusers
- ทุกการสุ่มกำหนด seed → รันซ้ำได้ผลใกล้เคียงเดิม (GPU อาจต่างเล็กน้อย) · เซลล์เทรน DDPM / แบบมีเงื่อนไข มีตัวกันรันซ้ำ
  - เวลาเทรนรวมราว 10–15 นาทีบน T4 (ประมาณการ) — ระหว่างรอให้ดูกราฟ loss และคุยว่า 'ควรเห็นอะไร'
  - ถ้า CUDA out of memory: Runtime → Restart แล้วรันใหม่ตั้งแต่เซลล์แรก (โมเดลวันนี้เล็กมาก ปกติไม่เกิด)




เปลี่ยน runtime เป็น T4 ก่อนเริ่ม

ตัวเลข/ภาพผลลัพธ์บนสไลด์มาจากการรัน notebook ฉบับเดียวกันบน CPU (ยังไม่ได้รันบน T4 จริง) — ผลบน T4 จะใกล้เคียงแต่ไม่เท่ากันเป๊ะ และเวลาจะสั้นกว่ามาก

*แผนภาพขั้นตอนใน Colab*

แล็บ 0

# เตรียมเครื่องมือ และดูข้อมูล

ติดตั้ง diffusers · โหลด Sprites 10,000 ภาพ 5 กลุ่ม · แปลงเป็น tensor ช่วง −1..1



*ผลจริงจากการรัน notebook แล็บ (CPU)*

แล็บ 0

# 0.1 ติดตั้งและตั้งค่า

```
!pip install -q -U "diffusers>=0.30" 2>/dev/null

import os, io, time, math, base64, zlib, warnings, random
import numpy as np, torch, torch.nn as nn, torch.nn.functional as F
import matplotlib.pyplot as plt, matplotlib.font_manager as fm
from PIL import Image
warnings.filterwarnings("ignore")

def set_seed(s=42):
    random.seed(s); np.random.seed(s); torch.manual_seed(s);
torch.cuda.manual_seed_all(s)
set_seed(42)
```

- **diffusers = ไลบรารี diffusion ของ Hugging Face (ให้ scheduler + U-Net)**
- PyTorch / torchvision มีใน Colab แล้ว
- **set_seed: random / numpy / torch / cuda ทั้งหมด**
  - ฟอนต์ Sarabun สำหรับกราฟ (ถ้าโหลดไม่ได้ก็ไม่ลัม)
  - ควรเห็น: อุปกรณ์: Tesla T4

แล็บ 0

# 0.2–0.3 โหลดข้อมูลและเตรียม tensor

```
x_all = torch.from_numpy(X).permute(0, 3, 1, 2).float() / 127.5 - 1.0
y_all = torch.from_numpy(Y)
print("x_all:", tuple(x_all.shape), "ช่วงค่า", x_all.min().item(), "ถึง",
x_all.max().item())

BATCH = 128
dataset = torch.utils.data.TensorDataset(x_all, y_all)
loader = torch.utils.data.DataLoader(dataset, batch_size=BATCH, shuffle=True,
drop_last=True)
print("จำนวน batch ต่อ epoch:", len(loader))
```

- **X: (10000, 16, 16, 3) uint8 · Y: (10000,) กลุ่ม 0–4**
- CLASS_NAMES = ฮีโร่ · ตัวประกอบ · อาหาร · คาถา · หันข้าง
- **x_all = X/127.5 − 1 → ช่วง −1..1 และสลับเป็น (N, 3, 16, 16)**
- BATCH = 128 → 78 batch ต่อ epoch
  - ภาพต่อกลุ่ม: [2000 2000 2000 2000 2000]
  - เซลล์ 0.2 เป็น form ซ่อนข้อความ base64 ยาว ๆ — กด ▶ ได้เลย

แล็บ 0

# ควรเห็น: Sprites 40 ภาพพร้อมชื่อกลุ่ม




- ตัวละครเกมพิกเซล 16×16 สีสด — กลุ่มเดียวกันมีท่าทางคล้ายกัน (ฮีโร่ยืนตรง, อาหารเป็นก้อน)
- 'โครงสร้าง' แบบนี้คือสิ่งที่ทั้ง 3 โมเดลต้องเรียน
- 2,000 ภาพต่อกลุ่ม สมดุลทุกกลุ่ม

แล็บ 1

# VAE — บีบภาพเป็นรหัส 16 ตัวเลข แล้วคลายกลับ

สร้างโมเดล → เทรน 15 epoch → คลายภาพ / สุ่มใหม่ / interpolate → แผนที่ latent



*ผลจริงจากการรัน notebook แล็บ (CPU)*

# 1.1 ทดสอบโมเดล 1 batch

```
    def decode(self, z):
        return self.dec(self.dec_fc(z).view(-1, 64, 4, 4))

    def forward(self, x):
        mu, logvar = self.encode(x); z = self.reparameterize(mu, logvar)
        return self.decode(z), mu, logvar, z

vae = VAE().to(device)
xb, yb = next(iter(loader)); xb = xb.to(device)
recon, mu, logvar, z = vae(xb)
print("ภาพเข้า", tuple(xb.shape), "→ z", tuple(z.shape), "→ ภาพออก", tuple(recon.shape))
```

- **forward คืน 4 อย่าง: recon, mu, logvar, z**
- **ควรเห็น: ภาพเข้า (128, 3, 16, 16) → z (128, 16) → ภาพออก (128, 3, 16, 16)**
- ภาพออกขนาดเท่าภาพเข้า — นี่คือ autoencoder
  - ตรวจ shape ก่อนเทรนเสมอ (นิสัยจากสัปดาห์ 1)

# 1.2 loss และการเทรน

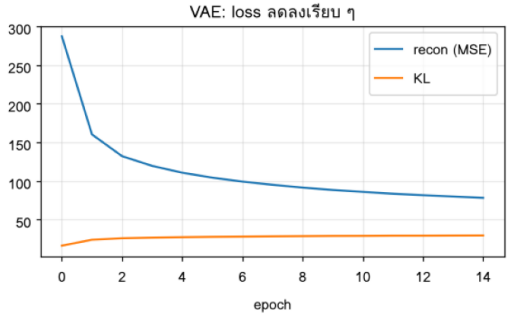
```
def vae_loss(recon, x, mu, logvar, beta=1.0):
    # ทั้งสองเทอมคิดเป็น "ต่อภาพ"
    recon_err = F.mse_loss(recon, x, reduction="sum") / x.size(0)
    kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp()) / x.size(0)
    return recon_err + beta * kl, recon_err, kl

VAE_EPOCHS = 15
opt = torch.optim.Adam(vae.parameters(), lr=1e-3)
hist = {"recon": [], "kl": []}
set_seed(0); t0 = time.time()
for ep in range(VAE_EPOCHS):
    vae.train(); s_r = s_k = 0.0
    for xb, _ in loader:
        xb = xb.to(device)
        recon, mu, logvar, _ = vae(xb)
        loss, r, k = vae_loss(recon, xb, mu, logvar, beta=1.0)
        opt.zero_grad(); loss.backward(); opt.step()
        s_r += r.item(); s_k += k.item()
```

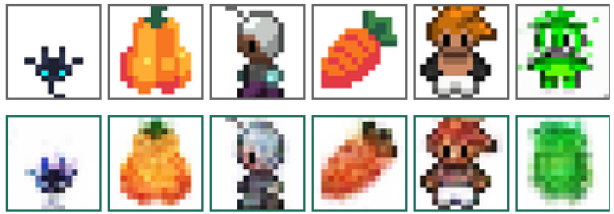
- **recon_err = MSE รวมทุกพิกเซล หารจำนวนภาพ**
- **kl = ค่าปรับที่บริเวณของภาพหนีจาก N(0, I)**
- beta = 1.0 (ลองเปลี่ยนเป็น 4 ดูในการบ้าน)
- VAE_EPOCHS = 15 · Adam lr 1e-3
  - ควรเห็น: recon ลดเร็ว, KL ค่อย ๆ นิ่ง — ไม่มีอะไรแกว่ง
  - ~1 นาทีบน T4 (ประมาณการ)

# ควรเห็น: loss ของ VAE ลดเรียบ และภาพคลายกลับเบลอ

loss ลดเรียบ (recon ลง, KL นิ่ง)




บน = ภาพจริง · ล่าง = VAE คลายกลับ



สีและท่าทางถูก แต่ขอบเบลอ

- อ่านผล: ภาพคลายกลับ 'จำโครงได้แต่เบลอ' — ตรงกับที่ทำนายไว้ตอนเช้า (MSE ชอบค่าเฉลี่ย)
- ตัวเลข/ภาพผลลัพธ์บนสไลด์มาจากการรัน notebook ฉบับเดียวกันบน CPU (ยังไม่ได้รันบน T4 จริง) — ผลบน T4 จะใกล้เคียงแต่ไม่เท่ากันเป๊ะ และเวลาจะสั้นกว่ามาก

# ควรเห็น: สุ่มภาพใหม่ และเดินใน latent

สุ่ม z ใหม่ 10 ชุด → ภาพที่ VAE สร้างเอง (ไม่มีในชุดข้อมูล)

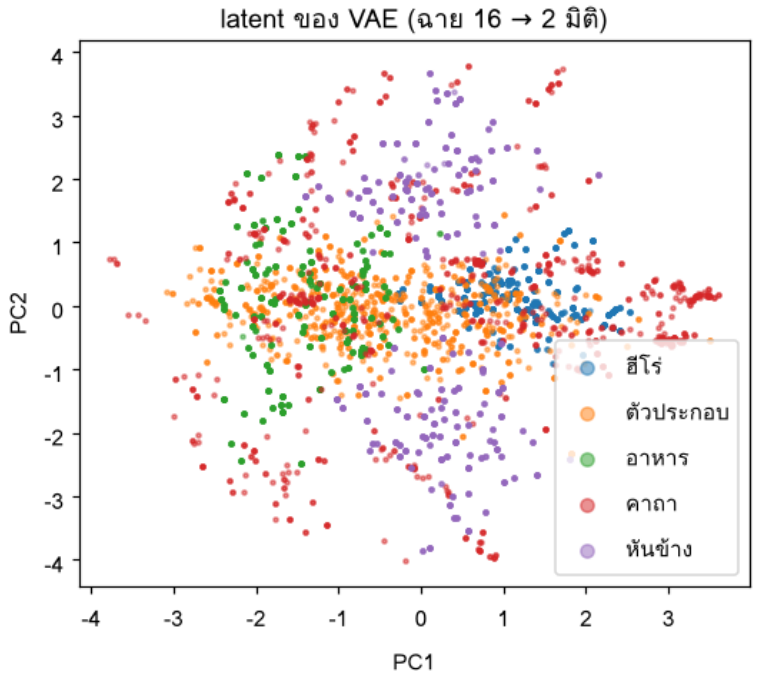


เดินเส้นตรงจาก sprite #0 ไป #7 ใน latent → ภาพค่อย ๆ เปลี่ยน ไม่กระโดด



- อ่านผล: สุ่ม z ตรงไหนก็ได้ภาพที่ 'ดูเป็นตัวละคร' เพราะ KL บังคับให้ latent ไม่มีรู
- interpolation ภาพค่อย ๆ morph ไม่กระโดด = latent ต่อเนื่อง

# ควรเห็น: แผนที่ latent แยกกลุ่มได้เอง




- encode ภาพทั้ง 10,000 → mu 16 มิติ → ฉายลง 2 มิติ (PCA)
- จุดสีเดียวกันเกาะกลุ่ม ทั้งที่ VAE ไม่เคยเห็นป้ายกลุ่มเลย
- ตำแหน่งใกล้กัน = ภาพคล้ายกัน — สัปดาห์หน้า Stable Diffusion ทำงานบน 'แผนที่' แบบนี้ (ใหญ่กว่า)

แล็บ 2

# GAN — เกมสองฝ่าย

สร้าง G, D → เทรน 25 epoch → ดู loss แกว่ง + ภาพทุก 5 epoch → เทียบกับ VAE + วัดความหลากหลาย



*ผลจริงจากการรัน notebook แล็บ (CPU)*

# 2.1 Generator และ Discriminator

```
class Generator(nn.Module):
    def __init__(self, nz=NZ):
        super().__init__()
        self.fc = nn.Linear(nz, 256 * 4 * 4)
        self.net = nn.Sequential(
            nn.BatchNorm2d(256), nn.ReLU(),
            nn.ConvTranspose2d(256, 128, 4, 2, 1), nn.BatchNorm2d(128), nn.ReLU(),   # 4→8
            nn.ConvTranspose2d(128, 64, 4, 2, 1), nn.BatchNorm2d(64), nn.ReLU(),     # 8→16
            nn.Conv2d(64, 3, 3, 1, 1), nn.Tanh())
    def forward(self, z): return self.net(self.fc(z).view(-1, 256, 4, 4))

class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(3, 64, 4, 2, 1), nn.LeakyReLU(0.2),                            # 16→8
            nn.Conv2d(64, 128, 4, 2, 1), nn.BatchNorm2d(128), nn.LeakyReLU(0.2),     # 8→4
            nn.Flatten(), nn.Linear(128 * 4 * 4, 1))      # 1 logit: บวก = จริง, ลบ = ปลอม
    def forward(self, x): return self.net(x)
```

- **G: z(64) → fc → 4×4×256 → 8×8 → 16×16 → Tanh**
- **D: 16×16 → 8×8 → 4×4 → Linear → 1 logit (บวก = จริง)**
- ทั้งคู่มี BatchNorm, D ใช้ LeakyReLU(0.2) — ทริกของ DCGAN ให้เทรนนิ่งขึ้น
  - D ออกเป็น logit (คะแนนดิบก่อน sigmoid) แล้วใช้ BCEWithLogitsLoss (นิ่งกว่า)
  - ควรเห็น: G(z): (128, 3, 16, 16) | D(ภาพ): (128, 1)

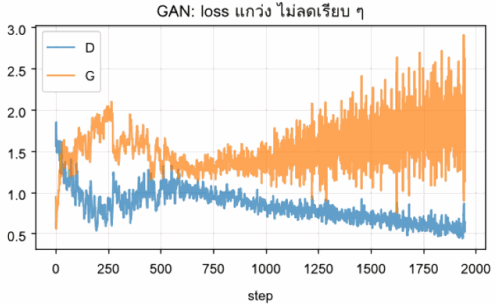
# 2.2 เทรนสลับสองขั้น

```
GAN_EPOCHS = 25
optD = torch.optim.Adam(D.parameters(), lr=2e-4, betas=(0.5, 0.999))
optG = torch.optim.Adam(G.parameters(), lr=2e-4, betas=(0.5, 0.999))
bce = nn.BCEWithLogitsLoss()
set_seed(0); z_fixed = torch.randn(16, NZ, device=device)
ghist, dhist, snapshots = [], [], []
t0 = time.time()
for ep in range(GAN_EPOCHS):
    for xb, _ in loader:
        xb = xb.to(device)
        real_lbl = torch.ones(BATCH, 1, device=device)
        fake_lbl = torch.zeros(BATCH, 1, device=device)
        # --- (1) ฝึกกรรมการ D ---
        fake = G(torch.randn(BATCH, NZ, device=device))
        lossD = bce(D(xb), real_lbl) + bce(D(fake.detach()), fake_lbl)
        optD.zero_grad(); lossD.backward(); optD.step()
        # --- (2) ฝึกคนปลอม G: อยากให้ D ตอบ "จริง" ---
        lossG = bce(D(fake), real_lbl)
        optG.zero_grad(); lossG.backward(); optG.step()
        ghist.append(lossG.item()); dhist.append(lossD.item())
```

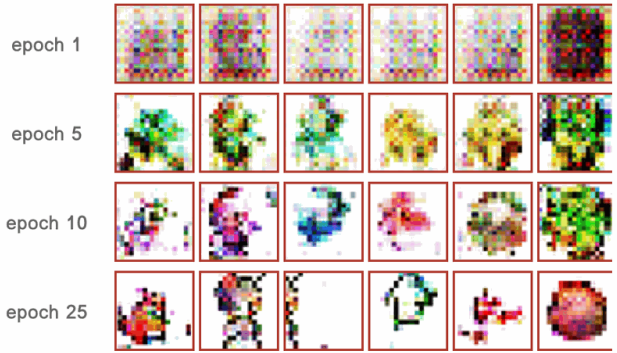
- **ขั้น (1) D: จริง→1, G(z).detach()→0**
- **ขั้น (2) G: D(G(z))→1 'โกหกตำรวจ'**
- z_fixed = noise 16 ชุดเดิม ไว้ดูพัฒนาการของ G
- Adam lr 2e-4 betas (0.5, 0.999) ทั้งคู่
  - GAN_EPOCHS = 25 · ~2 นาทีบน T4 (ประมาณการ)
  - ควรเห็น: lossD / lossG ขึ้น ๆ ลง ๆ ไม่ใช่ลดเรียบ

# ควรเห็น: loss แกว่ง และภาพจาก z_fixed ค่อย ๆ ชัด

loss ของ D และ G แกว่ง ไม่ลดเรียบ




ภาพจาก z_fixed ชุดเดิม ตามจำนวน epoch




- อ่านผล: loss ของ GAN บอกว่า 'ใครกำลังชนะ' ไม่ได้บอกว่าภาพดีแค่ไหน — ต้องดูภาพเสมอ

- ตัวเลข/ภาพผลลัพธ์บนสไลด์มาจากการรัน notebook ฉบับเดียวกันบน CPU (ยังไม่ได้รันบน T4 จริง) — ผลบน T4 จะใกล้เคียงแต่ไม่เท่ากันเป๊ะ และเวลาจะสั้นกว่ามาก

# 2.3 เทียบ VAE vs GAN + วัดความหลากหลาย

```
def diversity(imgs, n_pairs=2000, seed=0):
    """ระยะ L2 เฉลี่ยระหว่างภาพคู่สุ่ม (ภาพ tensor -1..1 รูปร่าง (N,3,16,16))"""
    g = torch.Generator().manual_seed(seed); flat = imgs.detach().cpu().flatten(1)
    i = torch.randint(0, len(flat), (n_pairs,), generator=g)
    j = torch.randint(0, len(flat), (n_pairs,), generator=g)
    return (flat[i] - flat[j]).norm(dim=1).mean().item()
```

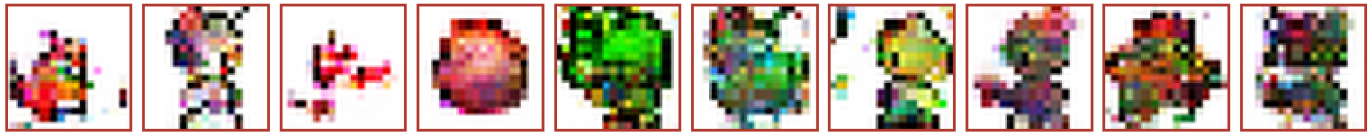
- **diversity = ระยะ L2 เฉลี่ยระหว่างภาพคู่สุ่ม 2,000 คู่**
- ยิ่งต่ำ = ภาพยิ่งคล้ายกัน: ซ้ำ (mode collapse) หรือเบลอ (VAE)
- **เทียบ 3 ค่า: ข้อมูลจริง 256 ภาพ · VAE 256 ภาพ · GAN 256 ภาพ**
  - ผลจากการรันบน CPU (ตัวอย่าง): ความหลากหลาย (ยิ่งสูงยิ่งหลากหลาย): ข้อมูลจริง 25.02 VAE 19.70 GAN 24.98
  - ตัวชี้วัดง่าย ๆ นี้แทน FID ที่จะเจอสัปดาห์หน้า

# ควรเห็น: GAN คมกว่า VAE ชัดเจน

VAE สุ่ม (เบลอ สีจาง)



GAN สุ่ม (คม สีสด)



- อ่านผล: ช่อง 'คุณภาพ' ของตารางตอนเช้าเห็นด้วยตา
- ช่อง 'ความหลากหลาย' ต้องวัด: GAN รอบนี้ยังไม่ collapse แต่เกมสองฝ่ายไม่รับประกันว่าจะเป็นแบบนี้ทุกรอบ — ลองเทรนต่อแล้ววัดซ้ำ

แล็บ 3

# DDPM — เดินจาก noise ทีละก้าว

forward noising → U-Net → เทรน 30 epoch → sample 1,000 ก้าว → DDIM 50 / 20 / 10 ก้าว



*ผลจริงจากการรัน notebook แล็บ (CPU)*

# 3.1 Forward: เติม noise ด้วย scheduler

```
from diffusers import DDPMScheduler, DDIMScheduler, UNet2DModel

T = 1000
# ตารางเติม noise แบบ cosine (เหมาะกับภาพเล็ก)
noise_sched = DDPMScheduler(num_train_timesteps=T, beta_schedule="squaredcos_cap_v2")

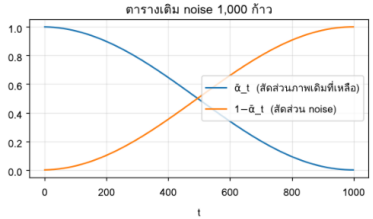
x0 = x_all[3:4].repeat(11, 1, 1, 1)        # sprite ตัวเดียว 11 ก๊อปปี้
ts = torch.tensor([0, 50, 100, 200, 300, 400, 500, 600, 700, 850, 999])
set_seed(0); noise = torch.randn_like(x0)
x_t = noise_sched.add_noise(x0, noise, ts)  # x_t = √ᾱ_t·x0 + √(1−ᾱ_t)·noise
```

- **DDPMScheduler(T=1000, 'squaredcos_cap_v2') = ตารางเติม noise แบบ cosine**
- **add_noise(x0, noise, t) = x_t = √ᾱ_t·x0 + √(1−ᾱ_t)·noise — กระโดดไป t ใดก็ได้**
- sprite ตัวเดียว 11 ก๊อปปี้ ที่ t ต่างกัน
  - cosine ค่อย ๆ เติมช้ากว่า linear ตอนต้น เหมาะกับภาพเล็ก
  - ควรเห็น: แถวภาพค่อย ๆ จม + กราฟ ᾱ_t

# ควรเห็น: ภาพจมใน noise และตารางเติม noise

sprite ตัวเดียวกัน ที่ t ต่าง ๆ — ค่อย ๆ จมใน noise




เส้นลง = ᾱ_t สัดส่วน 'ภาพเดิม' ที่เหลือ

เส้นขึ้น = สัดส่วน noise

t เล็ก = ลบ noise บาง ๆ · t ใหญ่ = ต้องจินตนาการโครงเอง

- อ่านผล: 'ระดับความยาก' ต่างกันตาม t — t เล็ก แค่ลบ noise บาง ๆ
- t ใหญ่ ต้องจินตนาการโครงเอง
- U-Net ตัวเดียวต้องทำได้ทุกระดับ (จึงรับ t เป็น input)

# 3.2 U-Net ตัวทาย noise (diffusers)

```
def make_unet(num_class_embeds=None):
    return UNet2DModel(
        sample_size=16, in_channels=3, out_channels=3, layers_per_block=1,
        block_out_channels=(64, 128, 128),
        # 16 → 8 → 4 มี self-attention ที่ชั้น 8×8 และ 4×4
        down_block_types=("DownBlock2D", "AttnDownBlock2D", "AttnDownBlock2D"),
        up_block_types=("AttnUpBlock2D", "AttnUpBlock2D", "UpBlock2D"),
        num_class_embeds=num_class_embeds)   # แล็บ 4 จะใส่ป้ายกลุ่มตรงนี้

unet = make_unet().to(device)
globals().pop("ddpm_losses", None)      # สร้างโมเดลใหม่ = ล้างสถานะ "เทรนแล้ว" ของเซลล์ 3.3
with torch.no_grad():
    out = unet(x_t.to(device), ts.to(device)).sample
```

- **UNet2DModel: in = out = 3 ช่อง, sample_size 16**
- block_out_channels (64, 128, 128): 16 → 8 → 4
- **Attn…Block2D = ใส่ self-attention ที่ชั้น 8×8 และ 4×4 (attention สัปดาห์ 2 บนพิกเซล)**
- num_class_embeds = None วันนี้ → แล็บ 4 ใส่ 6
  - เรียกใช้: unet(x_t, t).sample
  - ควรเห็น: ภาพเข้า (11, 3, 16, 16) → noise ที่ทาย (11, 3, 16, 16)

# 3.3 เทรน: เกมทาย noise

```
DDPM_EPOCHS = 30
if "ddpm_losses" in globals():
    print("เทรนไปแล้ว — ข้าม (ถ้าต้องการเทรนใหม่: รันเซลล์ 3.2 อีกครั้ง แล้วค่อยรันเซลล์นี้)")
else:
    ddpm_losses = train_diffusion(unet, DDPM_EPOCHS, tag="DDPM")
```

- **train_diffusion = ลูป 5 ขั้นตอนเช้า (ดูโค้ดเต็มใน notebook)**
- AdamW lr 2e-4 + cosine lr schedule
- **ตัวกันรันซ้ำ: ถ้ามี ddpm_losses แล้วจะข้าม — อยากเทรนใหม่ให้รันเซลล์ 3.2 (สร้าง unet ใหม่ + ล้างสถานะ) ก่อน**
- DDPM_EPOCHS = 30 · ~4 นาทีบน T4 (ประมาณการ)
  - ควรเห็น: loss ลดเรียบจาก ~0.4 ลงต่ำกว่า 0.1
  - ผลจากการรันบน CPU (ตัวอย่าง): [DDPM] epoch 30/30  loss 0.0762

# 3.4 Sampling: ฟังก์ชัน sample (ใช้ทั้งแล็บ 3 และ 4)

```python
@torch.no_grad()
def sample(model, scheduler, n=32, steps=None, labels=None, guidance=1.0, keep_every=None, seed=0):
    """สร้างภาพจาก noise · steps=None = ใช้ทุกก้าวของ scheduler · labels/guidance ใช้ในแล็บ 4"""
    model.eval(); g = torch.Generator(device=device).manual_seed(seed)
    scheduler.set_timesteps(steps or scheduler.config.num_train_timesteps)
    x = torch.randn(n, 3, 16, 16, device=device, generator=g); frames = []
    for i, t in enumerate(scheduler.timesteps):
        if labels is None:
            eps = model(x, t).sample
        else:
            eps_c = model(x, t, class_labels=labels).sample
            if guidance == 1.0: eps = eps_c
            else:
                null = torch.full_like(labels, 5)                    # ป้าย "ไม่บอกกลุ่ม"
                eps_u = model(x, t, class_labels=null).sample
                eps = eps_u + guidance * (eps_c - eps_u)             # classifier-free guidance
        x = scheduler.step(eps, t, x, generator=g).prev_sample
        last = i == len(scheduler.timesteps) - 1
        if keep_every and (i % keep_every == 0 or last): frames.append((int(t), x[0:1].clone()))
    return x, frames
```

- **scheduler.set_timesteps (steps) = เลือกว่าจะเดินกี่ก้าว**
- **ลูป: eps = model(x, t) → x = scheduler.step(eps, t, x).prev_sample**
- generator seed → noise ตั้งต้นเดิม = ภาพเดิม
  - labels / guidance ใช้ในแล็บ 4 (CFG) — วันนี้ None
  - keep_every เก็บภาพแรกระหว่างทางไว้ดูเส้นทาง

# ควรเห็น: ภาพโผล่จาก noise และภาพคมหลากหลาย




- อ่านผล: โครงใหญ่ (สี ท่าทาง) ถูกตัดสินตอน t สูง รายละเอียดมาตอน t ต่ำ
- ภาพคมเหมือน GAN และหลากหลายใกล้เคียง GAN ที่เทรนสำเร็จ — โดยไม่มีความเสี่ยง collapse และไม่ต้องจูน
- ตัวเลข/ภาพผลลัพธ์บนสไลด์มาจากการรัน notebook ฉบับเดียวกันบน CPU (ยังไม่ได้รันบน T4 จริง) — ผลบน T4 จะใกล้เคียงแต่ไม่เท่ากันเป๊ะ และเวลาจะสั้นกว่ามาก

# 3.5 DDIM: โมเดลเดิม เดินก้าวใหญ่ขึ้น

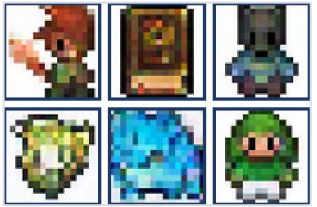
```
ddim_sched = DDIMScheduler.from_config(noise_sched.config)
rows = [("DDPM", 1000, t_ddpm, imgs_ddpm)]
for steps in [50, 20, 10]:
    t0 = time.time()
    imgs, _ = sample(unet, ddim_sched, n=32, steps=steps)
    rows.append(("DDIM", steps, time.time() - t0, imgs))

print(f"{'วิธี':6s} {'ก้าว':>5s} {'เวลา (s)':>9s} {'ความหลากหลาย':>14s}")
for name, steps, sec, imgs in rows:
    print(f"{name:6s} {steps:5d} {sec:9.1f} {diversity(imgs):14.2f}")
```

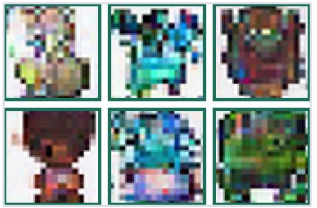
- **DDIMScheduler.from_config(noise_sched.config) = ตารางเดิม noise เดิม สูตรเดินใหม่**
- **ไม่ต้องเทรนอะไรใหม่ — unet ตัวเดิม**
- sample ด้วย 50 / 20 / 10 ก้าว จับเวลา + วัดความหลากหลาย
  - ควรเห็น: 50 ก้าว ≈ 1,000 ก้าว แต่เร็วกว่า ~20 เท่า · 10 ก้าวเริ่มเบลอ

# ควรเห็น: ตารางเวลา — ผลจากการรันบน CPU (ตัวอย่าง)

| วิธี | จำนวนก้าว | เวลา (s) — CPU | ความหลากหลาย |
|---|---|---|---|
| DDPM | 1000 | 82.0 | 22.89 |
| DDIM | 50 | 4.3 | 22.97 |
| DDIM | 20 | 1.8 | 23.02 |
| DDIM | 10 | 0.8 | 22.49 |



DDPM 1,000 ก้าว



DDIM 10 ก้าว

- ตัวเลขเวลาบนสไลด์มาจาก CPU — บน T4 ทุกแถวจะเร็วขึ้นหลายสิบเท่า แต่อัตราส่วนใกล้เคียงเดิม

*ผลจริงจากการรัน notebook แล็บ (CPU)*

# ควรเห็น: 1,000 ก้าว vs 50 / 20 / 10 ก้าว




- อ่านผล: ก้าวน้อยเกินไป = ลบ noise ก้าวละมาก ๆ จนเพี้ยน
- 20–50 ก้าวคือจุดคุ้ม
- sampler รุ่นใหม่ (DPM-Solver) ลงได้ถึง ~10 ก้าวโดยไม่เบลอ — สัปดาห์หน้าจะได้ลองใน Stable Diffusion

แล็บ 4

# Diffusion แบบมีเงื่อนไข — สั่งได้ว่าจะเอากลุ่มไหน

U-Net รับป้ายกลุ่ม → เทรน 30 epoch (ซ่อนป้าย 10%) → สั่งสร้างทีละกลุ่ม → หมุน guidance 1 / 3 / 6



*ผลจริงจากการรัน notebook แล็บ (CPU)*

# 4.1 เทรน U-Net แบบมีเงื่อนไข

```
    unet_c = make_unet(num_class_embeds=6).to(device)    # 0-4 = กลุ่ม, 5 = "ไม่บอกกลุ่ม"
    cond_losses = train_diffusion(unet_c, DDPM_EPOCHS, cond=True, p_uncond=0.1,
tag="cond")
```

- **num_class_embeds = 6: กลุ่ม 0–4 + ป้าย 5 = 'ไม่บอกกลุ่ม'**
- cond=True → ลูปเดิม แต่ป้อน class_labels=yb
- **p_uncond = 0.1: สุ่มซ่อนป้าย 10% ตอนเทรน → โมเดลเดียวทายได้ทั้งมี/ไม่มีเงื่อนไข (หัวใจของ CFG)**
  - มีตัวกันรันซ้ำเหมือนแล็บ 3 · ~4 นาทีบน T4 (ประมาณการ)
  - ควรเห็น: loss ลดเรียบใกล้เคียงแล็บ 3

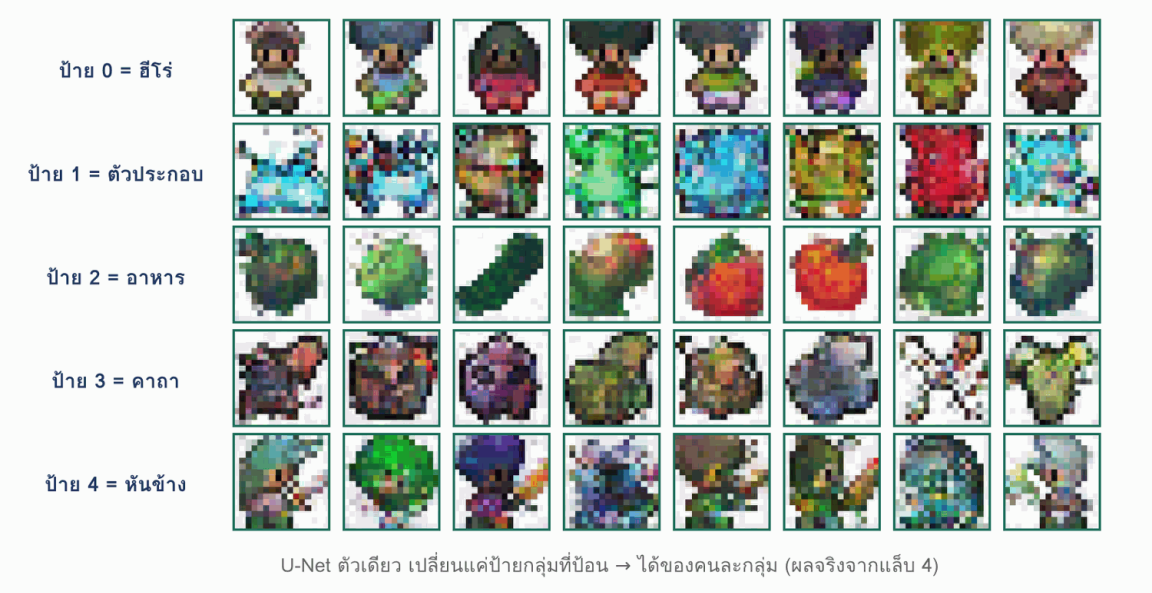
# 4.2 สั่งสร้างทีละกลุ่ม และหมุน guidance

```
labels = torch.arange(5, device=device).repeat_interleave(8)    # กลุ่ม 0..4 กลุ่มละ 8
imgs_c, _ = sample(unet_c, ddim_sched, n=40, steps=50, labels=labels, guidance=1.0)
show_grid(imgs_c, titles=[CLASS_NAMES[c] for c in labels.tolist()], nrow=8,
          suptitle="สั่งสร้างทีละกลุ่ม (guidance = 1)")

food = torch.full((16,), 2, device=device)    # กลุ่ม 2 = อาหาร
for w in [1.0, 3.0, 6.0]:
    imgs_w, _ = sample(unet_c, ddim_sched, n=16, steps=50, labels=food, guidance=w,
seed=7)
```

- **labels = 0,0,…,4,4 กลุ่มละ 8 → sample DDIM 50 ก้าว guidance = 1**
- **food = ป้าย 2 ทั้ง 16 ภาพ · seed เดิม (7) · guidance 1 / 3 / 6**
- ใน sample(): eps = eps_u + guidance × (eps_c − eps_u)
  - eps_u = ทายด้วยป้าย 5 ('ไม่บอก')
  - ควรเห็น: แต่ละแถวตรงกลุ่ม · guidance สูง → สีจัด ตรงกลุ่ม แต่ซ้ำกันมากขึ้น

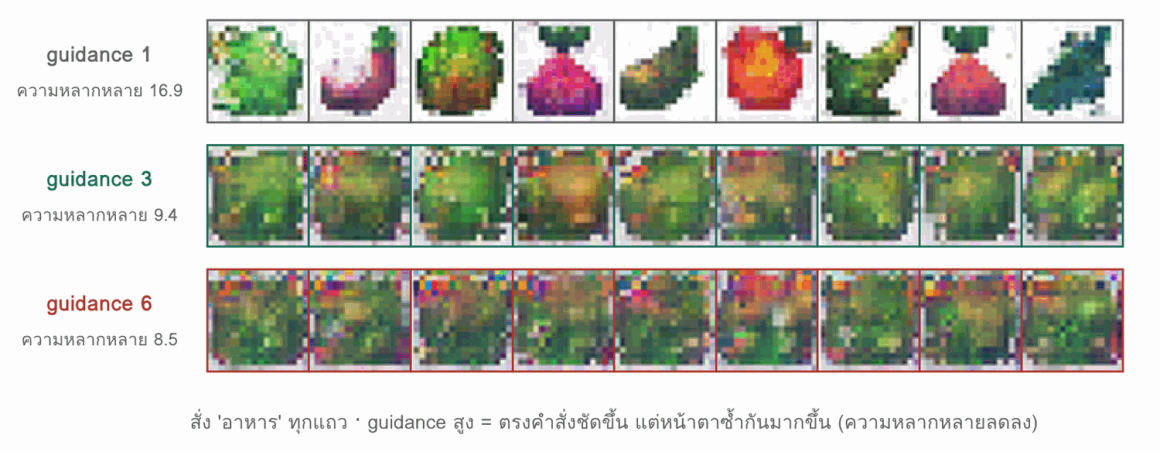
# ควรเห็น: สั่งสร้างทีละกลุ่ม (guidance = 1)




U-Net ตัวเดียว เปลี่ยนแค่ป้ายกลุ่มที่ป้อน → ได้ของคนละกลุ่ม (ผลจริงจากแล็บ 4)

- แถวละกลุ่ม (ฮีโร่ / ตัวประกอบ / อาหาร / คาถา / หันข้าง): U-Net ตัวเดียวสร้างได้ทุกกลุ่มตามป้ายที่สั่ง
- เทียบแล็บ 3 ที่สุ่มได้แต่เลือกกลุ่มไม่ได้
- กลไกเดียวกับ 'ข้อความ → ภาพ' สัปดาห์หน้า แค่เปลี่ยนป้ายเป็นเวกเตอร์จาก CLIP

# ควรเห็น: guidance สูง → ตรงกลุ่มขึ้น แต่หลากหลายลด




- อ่านผล: guidance = 'ขยายส่วนที่เงื่อนไขเปลี่ยน'
- สูงไป → สีจัด ซ้ำ เพี้ยน
- Stable Diffusion ใช้ค่าราว 5–8 — สัปดาห์หน้าจะได้หาค่าที่เหมาะเอง

สรุป

# สรุปแล็บ: 3 โมเดล ชุดข้อมูลเดียว — เห็นอะไรบ้าง

| | VAE (แล็บ 1) | GAN (แล็บ 2) | Diffusion (แล็บ 3–4) |
|---|---|---|---|
| loss ตอนเทรน | ลดเรียบ | แกว่ง | ลดเรียบ |
| ภาพที่ได้ | เบลอ | คม | คม |
| ความหลากหลาย (วัดได้) | ต่ำกว่าจริง (เบลอ) | ใกล้จริงเมื่อเทรนสำเร็จ — แต่ collapse ได้ | ใกล้ GAN ที่สำเร็จ — ทุกรอบ ไม่ต้องจูน |
| เวลาสร้าง 32 ภาพ | ทันที | ทันที | 1,000 ก้าวช้า → DDIM 50 ก้าว |
| สั่งกลุ่มได้ไหม | ไม่ (วันนี้) | ไม่ (วันนี้) | ได้ + หมุน guidance |



**VAE:** เบลอ



**GAN:** คม



**Diffusion:** คม หลากหลาย

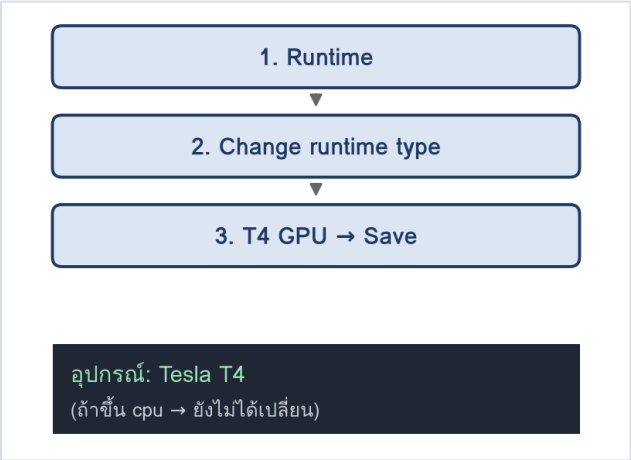
- สัปดาห์หน้า: VAE (แล็บ 1) + U-Net (แล็บ 3) + เงื่อนไขเป็นข้อความ (แล็บ 4) = Stable Diffusion ของจริง

*ผลจริงจากการรัน notebook แล็บ (CPU)*

แก้ปัญหา

# ปัญหาที่อาจเจอ และวิธีแก้

- **เซลล์แรกขึ้น 'อุปกรณ์: cpu' → Runtime → Change runtime type → T4 GPU แล้วรันใหม่ตั้งแต่ต้น**
- NameError (เช่น loader, show_grid, sample ไม่รู้จัก) → ข้ามเซลล์มา ให้ย้อนไปรันเซลล์ก่อนหน้า
- GAN พัง (lossD ≈ 0 ภาพเป็น noise ตลอด) → ลด lr ของ optD เป็น 1e-4 หรือรันเซลล์ 2.1 + 2.2 ใหม่ (seed เดิมควรไม่พัง)
- DDPM ไม่เทรนใหม่ทั้งที่แก้โค้ด → ตัวกันรันซ้ำ: รันเซลล์ 3.2 (สร้าง unet ใหม่และล้างสถานะ) แล้วค่อยรัน 3.3 · แล็บ 4: del cond_losses แล้วรัน 4.1 ใหม่
- ภาพผลต่างจากบนจอเล็กน้อย → ปกติถ้า GPU ต่างรุ่น (seed เท่ากันแต่การคำนวณไม่เหมือนกันเป๊ะ)
  - Colab ตัด session → ต้องเทรนใหม่ทั้งหมด (รวม ~15 นาที) — วันนี้โมเดลเล็ก ยอมรับได้; สัปดาห์หน้าจะสอนบันทึก weight ลง Drive




เกือบทุกปัญหา: เช็ก runtime แล้วรันใหม่จากบนสุด

ชุดข้อมูลและโค้ดทั้งหมดอยู่ในไฟล์เดียว ถ้าพังให้ Restart runtime แล้วรันใหม่จากบนสุด

*แผนภาพขั้นตอนใน Colab*

การบ้าน

# การบ้านสัปดาห์ 4: "3 โมเดล 1 ชุดข้อมูล"

### 1 โจทย์

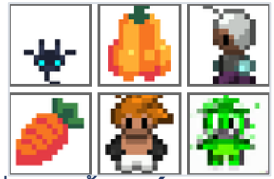
เลือกชุดภาพเล็ก ๆ ของตัวเอง (≥ 2,000 ภาพ, ย่อเป็น 16×16 หรือ 32×32)
เทรน VAE / GAN / DDPM ด้วยโค้ดวันนี้ (HW4_Starter.ipynb)
เทียบ: ภาพ · ความหลากหลาย · เวลา

### 2 ทดลองเพิ่ม 1 อย่าง

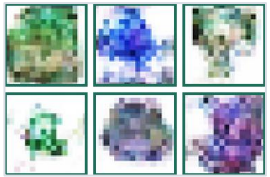
เลือก 1: β ของ VAE (1 vs 4) · DDIM step sweep (100/50/20/10) · guidance sweep (ถ้ามีป้าย) · จำนวน epoch ของ GAN
รายงานเป็นตาราง + ภาพ

### 3 ส่ง

notebook ที่รันแล้ว + my_images.npy + hw4_results.json + memo 1 หน้า (โมเดลไหนเหมาะกับข้อมูลของคุณเพราะอะไร) + สไลด์ 2 หน้า
ส่งก่อนคาบสัปดาห์ 5 · นำเสนอ 3 นาที
รายละเอียดและเกณฑ์ในเอกสารรายวิชา



ตัวอย่างชุดข้อมูล (ของคุณจะต่างไป)



VAE



GAN



DDPM

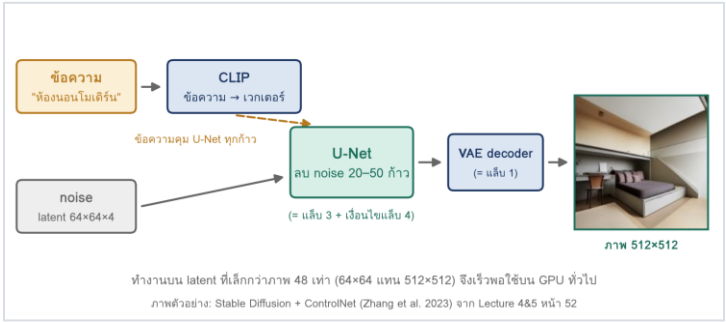
*ผลจริงจากการรัน notebook แล็บ (CPU)*

ปิดวัน

# สัปดาห์หน้า

**เอา VAE (แล็บ 1) + U-Net (แล็บ 3) + เงื่อนไข (แล็บ 4) มาต่อกันเป็น Stable Diffusion**
**แล้วเรียนวิธีควบคุม (img2img, ControlNet) และปรับแต่งเป็นของเรา (LoRA)**

คำถามที่ทิ้งไว้ตอนเช้า: 'สร้างภาพสินค้าของบริษัทตัวเอง' ควรเทรนตั้งแต่ศูนย์ หรือปรับโมเดลที่มีอยู่? — สัปดาห์หน้าได้คำตอบด้วยมือ




*แผนภาพ Stable Diffusion (ภาพห้องนอนจาก Siyu Zhu Lecture 4&5 หน้า 52)*